

JavaScript DOM & Data

Mini-chantier E-commerce Analytics

Charger, afficher, filtrer, trier et analyser des données JSON avec JavaScript vanilla.

Niveau	beginner+ vers intermediate
Technologies	HTML, CSS léger, JavaScript vanilla, DOM, fetch, JSON
Bonus	Chart.js, uniquement en challenge optionnel
Format	3 exercices liés + 1 challenge optionnel
Données	3 fichiers JSON fournis séparément

Le chantier en deux minutes

CONTEXTE

Tu construis un petit tableau de bord d'analyse pour une boutique en ligne. Les données existent déjà : trois fichiers JSON te sont fournis. Ton travail consiste à les charger dans le navigateur, les afficher, les rendre explorables, puis en tirer quelques indicateurs utiles.

CE QUE TU VAS FAIRE

- charger des données dans le navigateur ;
- afficher une table HTML générée depuis les données ;
- rechercher, filtrer et trier ;
- calculer des indicateurs ;
- afficher les résultats dans le DOM ;
- éventuellement visualiser des données avec Chart.js.

CE QUI N'EST PAS DEMANDÉ

- pas de backend ;
- pas d'authentification ;
- pas de tunnel de commande ;
- pas de paiement ;
- pas de panier complet ;
- pas de maquette Figma ;
- pas d'application e-commerce complète.

Ce document ne contient aucune solution : les indices servent à débloquer, pas à remplacer ton code.

Fichiers sources

Trois fichiers JSON sont fournis séparément. Leur structure n'est pas décrite ici : ouvre-les, observe-les et déduis toi-même les champs disponibles avant de coder.

Fichier	Description
products.json	Catalogue produit de la boutique.
orders.json	Commandes passées, avec le détail des lignes de chaque commande.
customers.json	Clients de la boutique.

L'analyse des données fait partie du travail : identifier les champs utiles, repérer les données imbriquées et comprendre comment relier les fichiers entre eux.

Exigences transverses

CODE

- JavaScript vanilla uniquement ;
- `const` et `let`, jamais `var` ;
- console sans erreur ;
- aucune donnée recopiée à la main dans le HTML.

ORGANISATION DU PROJET

L'arborescence et le découpage des fichiers sont libres : c'est à toi de proposer une structure qui tient. On attend simplement qu'elle soit lisible et justifiable, et que les responsabilités soient séparées : chargement, transformation des données, calculs, rendu.

BEST PRACTICES ATTENDUES

- KISS : la solution la plus simple qui fonctionne, avant toute optimisation ;
- DRY : pas de logique dupliquée d'un endroit à l'autre ;
- SOLID, en particulier la responsabilité unique : une fonction fait une seule chose et la fait bien ;
- noms explicites, fonctions courtes, effets de bord limités.

Exercice 01 — Charger les produits et afficher une table

beginner+

CONTEXTE

Première étape du tableau de bord : faire apparaître le catalogue dans la page. Rien n'est écrit à la main dans le HTML, tout vient du fichier JSON.

NOTIONS JS TRAVAILLÉES

`fetch`, `async / await`, `JSON`, `map`, `template literals`, sélection et modification du DOM, `try / catch`.

CE QUI DOIT ÊTRE COUVERT

- le chargement de `products.json` ;
- l'affichage d'une table générée depuis les données ;
- les trois cas d'interface : chargement, erreur, liste vide ;
- le nombre total de produits chargés.

CONTRAINTES TECHNIQUES

- utiliser `fetch` ;
- utiliser `async / await` ;
- ne pas modifier `products.json` ;
- ne pas écrire les lignes de la table à la main ;
- garder le HTML initial léger.

INDICES PROGRESSIFS

1. Commence par afficher le résultat du `fetch` dans la console, avant de toucher au DOM.
2. Une fonction peut recevoir un tableau et retourner une chaîne HTML.
3. Si la table reste vide, vérifie le chemin du fichier JSON et le serveur local.

Bonus (optionnel) · Ajouter une classe CSS différente si le stock est faible, pour repérer les produits à surveiller.

À RETENIR

- `fetch` récupère une donnée de manière asynchrone ;
- le DOM permet d'afficher les données dans la page ;
- les données ne doivent jamais être recopiées à la main dans le HTML.

Exercice 02 — Rechercher, filtrer et trier le catalogue

intermediate

CONTEXTE

Le catalogue s'affiche, mais il n'est pas encore explorable. Tu ajoutes maintenant les contrôles qui permettent de retrouver un produit et de réordonner la liste.

NOTIONS JS TRAVAILLÉES

`addEventListener`, `filter`, `sort`, `toLowerCase`, copie de tableau avec `[...tableau]`, notion d'état d'interface.

CONTRÔLES ATTENDUS

- champ de recherche par nom ;
- filtre par catégorie ;

- filtre stock : tous / en stock / stock faible ;
- tri par prix : aucun / croissant / décroissant ;
- bouton reset ;
- compteur de résultats visibles.

CONTRAINTES TECHNIQUES

- utiliser `addEventListener` ;
- utiliser `filter` ;
- utiliser `sort` ;
- utiliser `toLowerCase` pour la recherche ;
- les filtres doivent pouvoir se combiner ;
- éviter de modifier directement le tableau original lors du tri ;
- ne pas créer plus de trois filtres.

INDICES PROGRESSIFS

1. Commence par une seule recherche, puis ajoute les autres filtres un par un.
2. Une fonction du type `getVisibleProducts()` peut retourner la liste filtrée puis triée.
3. Pour trier sans modifier l'original, copie d'abord le tableau avec `[...products]`.

Bonus (optionnel) · Afficher un message clair si aucun produit ne correspond aux filtres en cours.

À RETENIR

- une interface dynamique suit souvent le cycle : état, transformation des données, rendu ;
- les filtres doivent pouvoir se combiner entre eux ;
- un tri peut modifier le tableau d'origine si on ne le copie pas.

Exercice 03 — Calculer et afficher des KPI de commandes

intermediate

CONTEXTE

La table produits fonctionne. Tu passes maintenant aux commandes pour produire les chiffres qui résument l'activité de la boutique.

NOTIONS JS TRAVAILLÉES

reduce, parcours de données imbriquées, fonctions pures, arrondi des montants, composition de petites fonctions.

KPI ATTENDUS

- nombre total de commandes ;
- chiffre d'affaires total ;
- panier moyen ;
- nombre de commandes livrées ;
- nombre de commandes en attente ;
- taux de livraison.

CONTRAINTES TECHNIQUES

- utiliser `reduce` au moins une fois ;
- ne pas faire les calculs directement dans le HTML ;
- créer des fonctions séparées pour les calculs principaux ;
- arrondir les montants de manière lisible ;
- ne pas modifier `orders.json`.

INDICES PROGRESSIFS

1. Le total d'une commande dépend de toutes ses lignes.
2. Le total d'une ligne se déduit de la quantité et du prix unitaire.
3. Le panier moyen est le chiffre d'affaires total divisé par le nombre de commandes.

Bonus (optionnel) · Afficher le chiffre d'affaires par statut, puis les trois produits les plus vendus si le catalogue est déjà chargé.

À RETENIR

- `reduce` transforme une liste en une valeur ou en un objet de synthèse ;
- un KPI se calcule à partir des données, il ne s'écrit jamais en dur ;
- séparer calcul et rendu rend le code beaucoup plus facile à vérifier.

Challenge optionnel — Visualiser les données avec Chart.js

intermediate+

CONTEXTE

Le socle fonctionne : tu peux maintenant traduire un calcul en graphique. À traiter seulement si les trois exercices précédents tiennent debout.

CONSIGNE

Choisir au moins un graphique parmi les suivants :

- doughnut chart des commandes par statut ;

- bar chart du chiffre d'affaires par catégorie ;
- line chart du chiffre d'affaires par date.

CHALLENGE SUPPLÉMENTAIRE

- relier les commandes aux clients ;
- relier les lignes de commande aux produits ;
- calculer le chiffre d'affaires par ville ou par catégorie.

CONTRAINTES TECHNIQUES

- Chart.js reste optionnel ;
- ne pas casser les exercices 01 à 03 ;
- aucune valeur écrite en dur dans le graphique ;
- expliquer en commentaire court quelles données alimentent le graphique.

INDICES PROGRESSIFS

1. Chart.js attend souvent deux choses : des labels et un ou plusieurs datasets.
2. Avant de créer le graphique, affiche les labels et les valeurs dans la console.
3. Si rien ne s'affiche, vérifie le CDN, la présence du canvas et l'ordre de chargement des scripts.

À RETENIR

- une visualisation est la traduction graphique d'un calcul ;
- préparer les données est souvent plus important que créer le graphique ;
- une librairie externe s'intègre seulement quand la logique de base fonctionne déjà.

Résultat attendu du chantier

Un seul résultat est évalué : le tableau de bord dans son ensemble.

- le projet se lance dans un navigateur et affiche un tableau de bord fonctionnel ;
- les produits et les commandes proviennent des fichiers JSON fournis, jamais du HTML ;
- la table produits est explorable : recherche, filtres combinables, tri, compteur juste ;
- les KPI des commandes sont visibles et cohérents avec les données ;
- l'interface reste lisible et compréhensible, y compris quand aucun résultat ne correspond ;
- la console ne contient aucune erreur ;
- le code montre une séparation claire entre chargement, transformation, calcul et rendu ;
- le challenge Chart.js, s'il est réalisé, ne casse rien du socle.

Livrables

- le lien du dépôt ou le dossier du projet, selon la consigne donnée ;
- une page HTML fonctionnelle et les fichiers JavaScript du projet ;
- un README court : comment lancer le projet, et ce qui est fait ou non.

Bon chantier. Avance par petites étapes, garde la console ouverte, et fais fonctionner le socle avant de penser au bonus.